

Inhaltsverzeichnis

1. Welcome	1
2. Fachlicher Schwerpunkt des Kurses (Zugdisposition)	1
2.1. Kontext	1
2.2. Vorschau: Klassenmodell	2
3. Prozess der Softwareentwicklung	4
4. IDE Setup	5
4.1. Lokales Arbeitsverzeichnis: Der Workspace	5
4.2. Java Version	5
4.3. IDE IntelliJ	6
4.4. Quellcodeverwaltung: Git	7
4.5. Herunterladen der Kursinhalte (Git mit IntelliJ)	8

1. Welcome

Über mich



Thorsten Eckstein

Konzern	Deutsche Bahn, ca. 335.000 Mitarbeiter DB Systel GmbH, Frankfurt a.M., ca. 7.000 Mitarbeiter, IT-Dienstleister der Deutschen Bahn
Qualifikationen	Dipl. Geo-Informatik, zertifizierter Softwareentwickler, zertifizierter Softwarearchitekt

2. Fachlicher Schwerpunkt des Kurses (Zugdisposition)

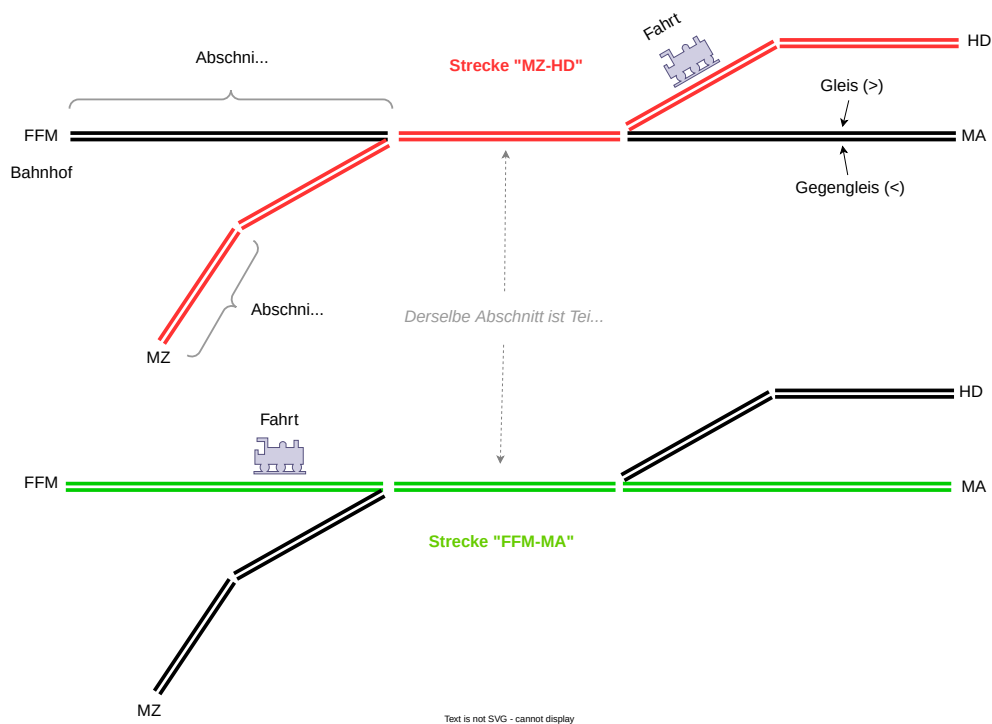
2.1. Kontext

Der **Betrieb** von Zügen kann grundsätzlich unterteilt werden in:

1. **infrastrukturelle** Sicht (*Stammdaten*)
2. **planerische** Sicht (*Bewegungsdaten, SOLL*)

3. operative Sicht (Bewegungsdaten, IST)

Ein paar wichtige Begriffe:



2.2. Vorschau: Klassenmodell

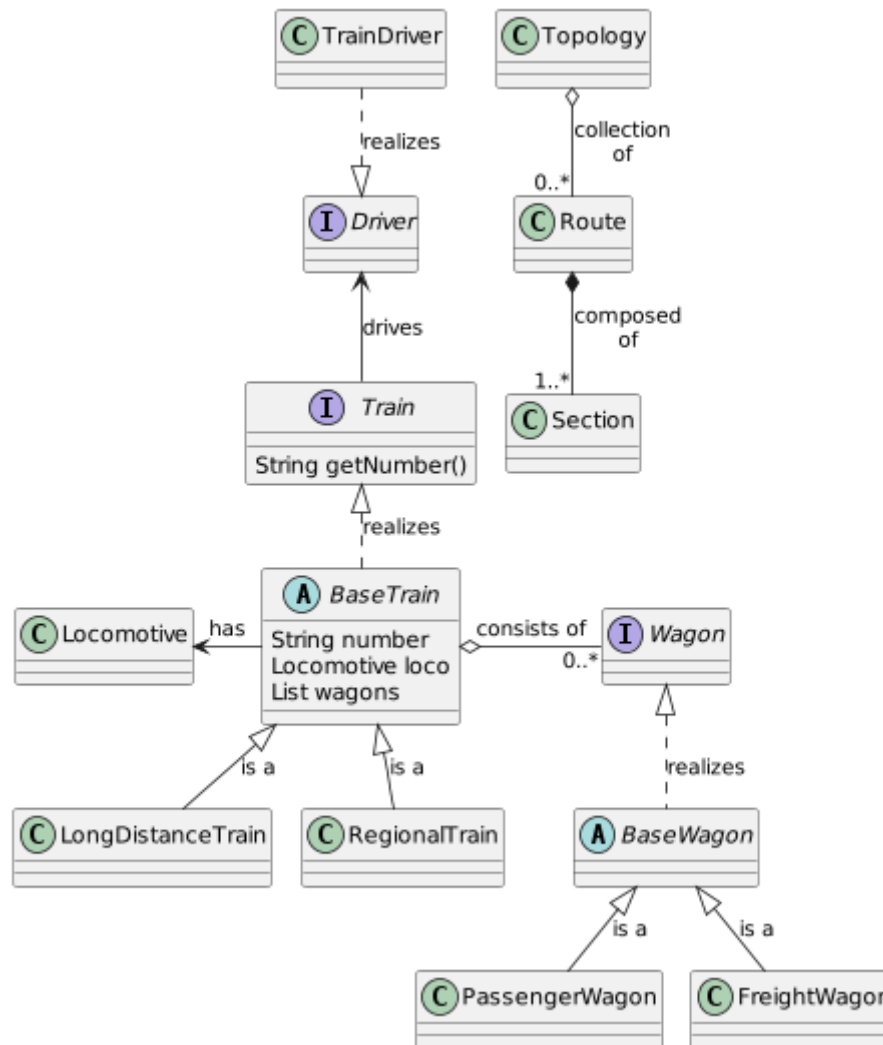
Eine solche Fachlichkeit (s.o.) muss, um Applikationen zu erstellen, in (Fach-) **Klassenmodellen** abgebildet werden, z.B. mit folgenden Objekten:

- + Zug (Zugarten), Wagon (Wagonarten)
- + Strecke, Abschnitt, Gleis
- + Ereignisse: Abfahrt, Ankunft
- + Fahrt
- + Fahrplan
- + Fahrgast
- + Dispositionsmaßnahme, Umleitung

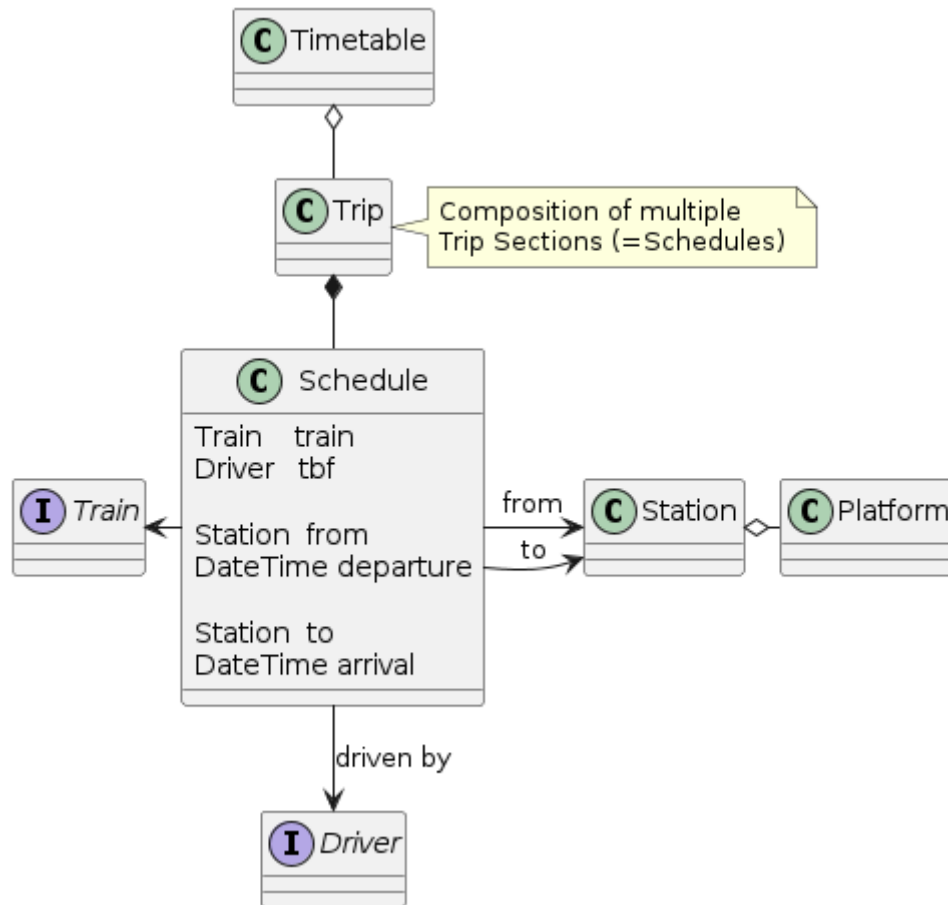
und viele weitere ...

Beispiel für ein grundsätzliches **Fachklassenmodell** mit zwei "Geschmacksrichtungen":

(1) **Infrastrukturelle** Sicht (statische Sicht):



(2) **Planerische** Sicht (dynamische Sicht):



3. Prozess der Softwareentwicklung

Der **agile Entwicklungsprozess** (agile Software Development) kann am besten mittels Grafik erläutert werden:

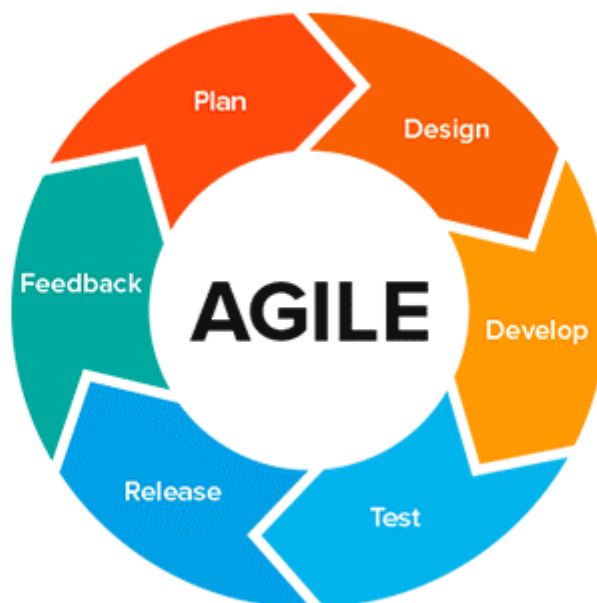


Figure 1. Der agile Entwicklungsprozess

In allen Teilschritten des Prozesses werden viele verschiedene **Tools** eingesetzt, dazu gehören auch

diese, die in dieser LV eingesetzt werden, vor allem:

- Entwicklungsumgebung **IntelliJ** (v.a. in Phasen "Design", "Develop" und "Test") sowie
- Quellcodeverwaltung **Git** (eigentlich in allen Phasen)

4. IDE Setup

4.1. Lokales Arbeitsverzeichnis: Der **Workspace**

Zuerst wird ein **lokaler Workspace** eingerichtet, d.h. ein (separates) Wurzelverzeichnis, in das im Anschluss das die **Inhalte des Kurses**, d.h. SourceCode-Repository **gespeichert** werden können.



Natürlich kann man hier immer den persönlichen Präferenzen für dieses Wurzelverzeichnis folgen, für die Einrichtung des Kurses ist es aber besser, den Empfehlungen zu folgen ...

Der Workspace - das **lokale Arbeitsverzeichnis**:

Zur Einrichtung eines Workspace unter **Windows** (11) nutzt man am besten einfach den **Explorer** und erzeugt ein neues Verzeichnis, zum Beispiel (wird auch in späteren Beispielen genutzt) das folgende:

```
D:\Projekte
```

Wie gesagt, hierin werden die Kursinhalte in den nächsten Schritten dann lokal gespeichert und verwaltet (→ mittels **git**, siehe zugehöriges Kapitel).

4.2. **Java** Version

Empfehlung: Im Kurs soll folgende **Java** Version verwendet werden:

```
21
```

Ggf. muss diese Version noch installiert werden, wobei die Versionsnummern ("Minor") *hinter* der Hauptversionsnummer 21 ("Major") nicht so wichtig sind.

Eine von mehreren Möglichkeiten, die aktuelle genutzte Java Version herauszufinden, ist:

- In den neuen Workspace, momentan noch leeren Ordner (s.o.), wechseln

```
cd Projekte
```

- Hier ein Terminal/Kommando-Shell öffnen
- Prüfen, ob **java** vorhanden ist, einfach durch Eingabe von

```
java -version
```

Dann sollte eine Java Version angezeigt werden.

Eine andere ist einfach innerhalb von IntelliJ, **nachdem** wir die LV-Inhalte bzw. das Gitlab-Repository heruntergeladen haben.

Ist dies erfolgt, kann man einfach hier schauen:

```
Datei > Projektstruktur > SDKs
```

4.3. IDE **IntelliJ**

Die empfohlene IDE ist **IntelliJ**. Die LV-Inhalte sollten sowohl in der *Community* als auch in der *Ultimate* Edition funktionieren.

Der Code für den Kurs wurde mit folgender Version erstellt, die installiert sein sollte (falls noch nicht vorhanden, bitte installieren):

```
IntelliJ IDEA 2024.3.1
```

Kleinere Abweichungen in der Version sollten kein Problem sein, die letzten Versionen sind in aller Regel untereinander kompatibel. Bei Problemen versucht man es einfach mit der aktuellsten IDE Version.



Alternativ könnte auch **MS Visual Studio Code** genutzt werden und diese Umgebung sollte auch bei der Einrichtung keine besonderen Probleme verursachen. Aber ... auf eigene Gefahr ;-), die Lehrveranstaltung wurde für IntelliJ vorbereitet!

IntelliJ **PlugIns**

Die PlugIns von IntelliJ finden sich hier über das Haupt-Menü:

```
IntelliJ IDEA > Settings > PlugIns
```

Bundled PlugIns

Mit IntelliJ werden bereits diverse PlugIns **automatisch mitinstalliert**, zu diesen gehören die folgenden. Sie sind für das Seminar erforderlich und sollen anfangs mal geprüft werden (Menü **IntelliJ > Settings > PlugIns**):

PlugIn Name	Kommentar
Git & GitHub	<i>Version Control: Source Code Management, lokaler Git-Client</i>
Maven	<i>Build Tools: Build, Dependency & Project Management</i>

Non-bundled PlugIns

Für den Quellcode, Tests und Dokumentation sind weitere PlugIns erforderlich, die *nicht* mit IntelliJ installiert werden, also separat installiert werden müssen:

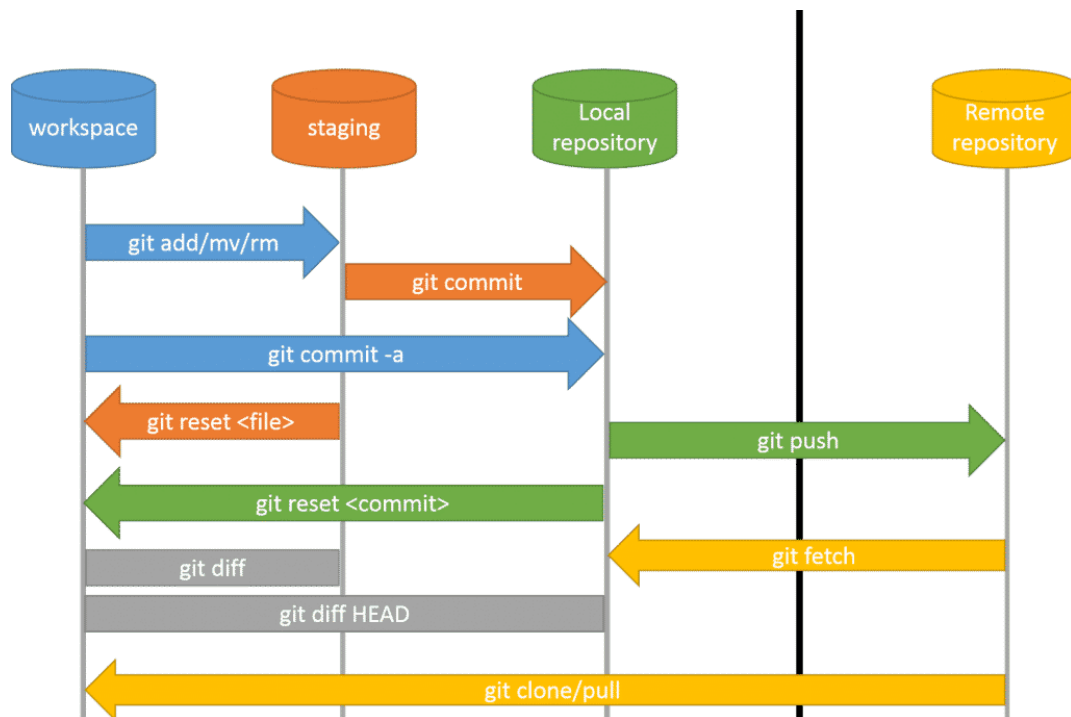
PlugIn Name	Was ist das?	aktuelle Version
 AsciiDoc  0.38.9 AsciiDoctor IntelliJ Plugin Project	<i>Einheitliche Erstellung von Dokumentationen, z.B. als PDF</i>	<i>>= 0.43.x</i>
 Diagrams.net Integration  0.1.14 Docs As Code	<i>Erstellung von beliebigen Diagrammtypen</i>	<i>>= 0.2.6</i>
 PlantUML Integration  5.22.0 Eugene Steinberg, Vojtech Krasa	<i>Erstellung von UML-Diagrammen</i>	<i>>= 7.11.x</i>

PS: Ich persönlich aktualisiere IntelliJ selbst und auch die PlugIns in der Regel recht zeitnah.

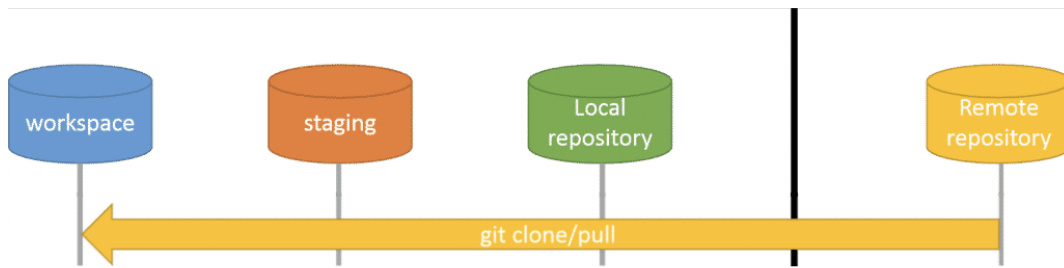
4.4. Quellcodeverwaltung: Git

Web: <https://de.wikipedia.org/wiki/Git>

Git ist ein sogenanntes *verteiltes* Versionsverwaltungssystem für Quellcode und setzt folgenden **Workflow** um, bzw. bietet folgende Funktionen:



In dieser LV kommt aber nur ein kleiner Teil hiervon zum Einsatz, lediglich das erstmalige **Downloaden** (*clone*) & die regelmäßige **Aktualisierung** (*pull*) der lokalen Dateien wird benötigt:



Das lokale Arbeitsverzeichnis wird **workspace** genannt. In diesem werden alle Quellcode-Dateien gespeichert, also z.B. das Verzeichnis, in das der Quellcode heruntergeladen wird:

In Windows bspw.:

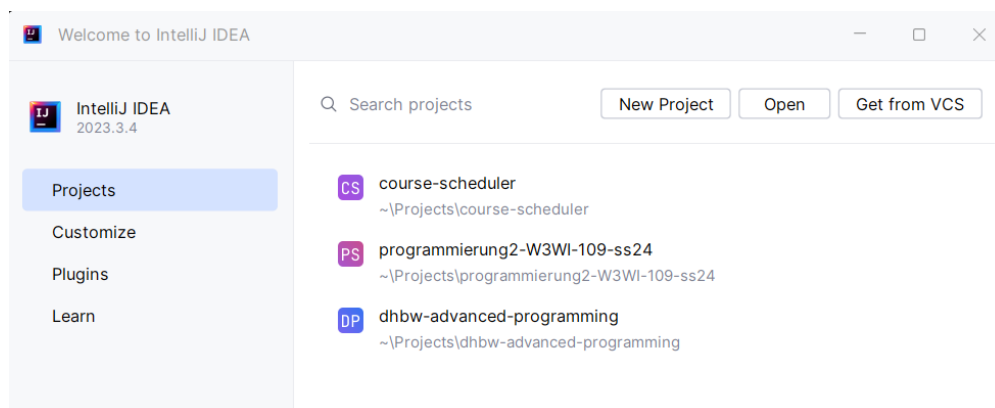
D:/Projekte

oder unter MacOS im Benutzerverzeichnis (aktueller Benutzer hier **<name>**)

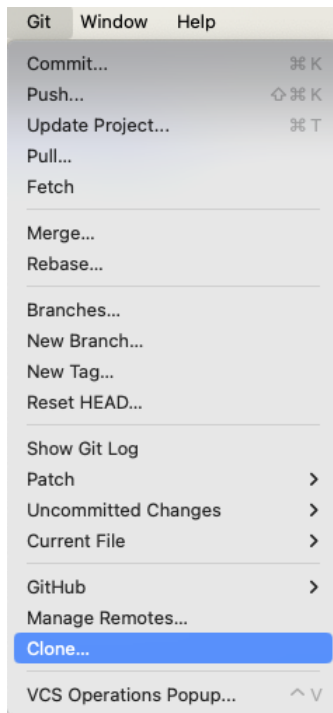
/Users/<name>/Projects

4.5. Herunterladen der Kursinhalte (Git mit IntelliJ)

1. Über den Explorer/Finder in das eigene *Workspace*-Verzeichnis **wechseln**, z.B. **D:\Projekte**
2. **IntelliJ starten**. Wenn zuvor kein anderes Projekt geladen war, dann kommt ...

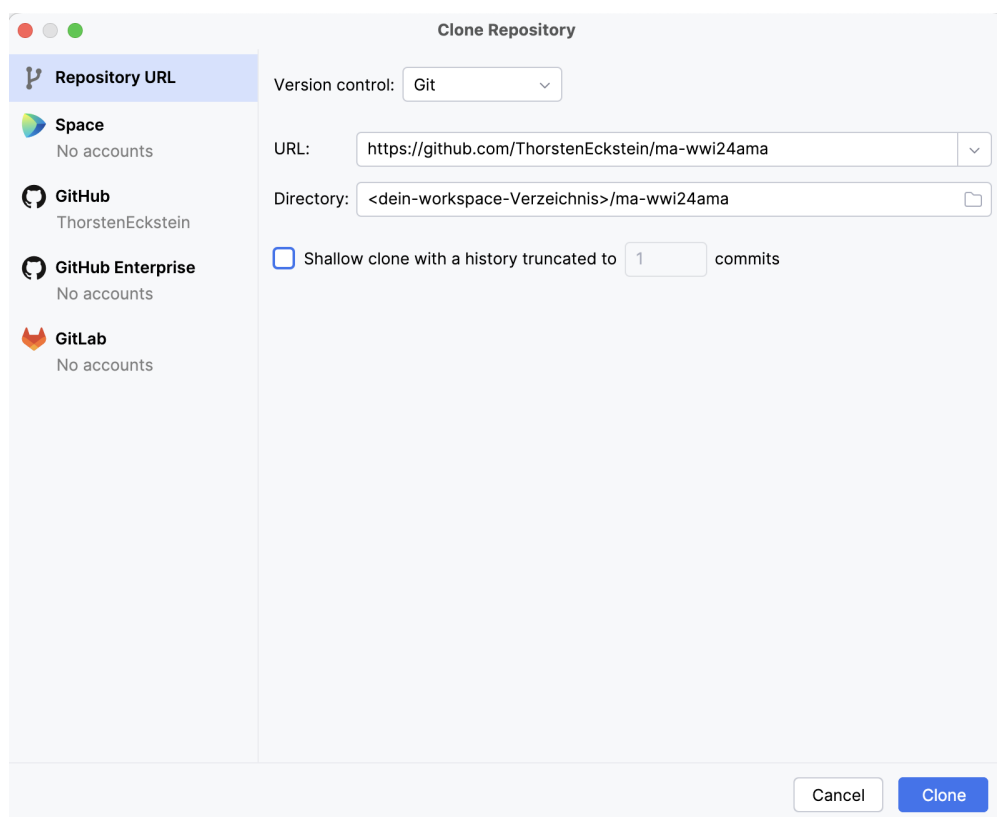


3. Nun soll das Projekt von GitHub **heruntergeladen** werden, dazu gibt es 2 Varianten:
 - a. **Clone Repository**, in älteren Versionen **Get from VCS** anklicken (VCS = Version Control System, **Git**)
 - b. oder über die Menüoption **Git > clone ...** aufrufen (*falls bereits ein anderes Projekt in IntelliJ automatisch geladen wurde*)

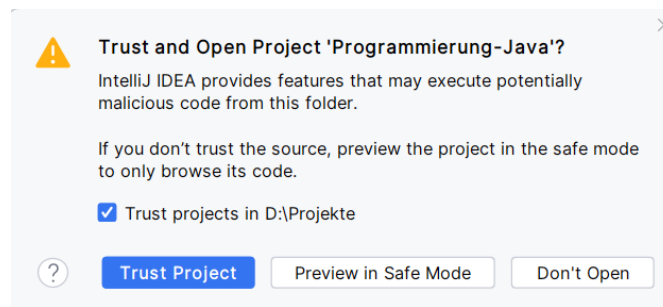


4. In beiden Fällen erscheint der gleiche **clone-Dialog**, in dem die folgenden Angaben eingegeben werden müssen:

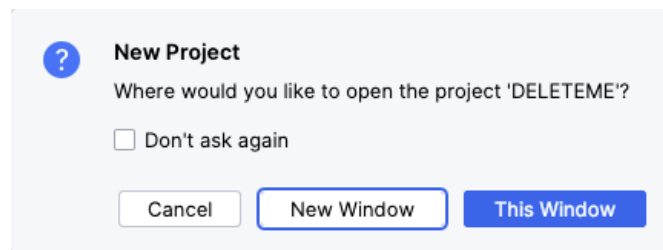
URL : <https://github.com/ThorstenEckstein/ma-wwi24ama>
 Workspace : <dein-lokales-Arbeitsverzeichnis>/ma-wwi24ama



5. Dann kommt (wahrscheinlich) eine Abfrage zur **"Vertrauenswürdigkeit"** → [Trust Project]:



6. Gegenenfalls muss noch entscheiden, ob das Projekt **in einem separaten Fenster** geöffnet werden soll → **[New Window]**:



Nun sollte das heruntergeladene Projekt in IntelliJ vorhanden sein.

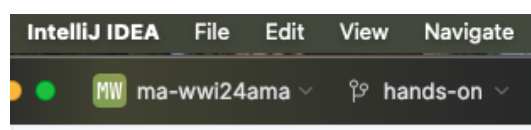


Falls irgendetwas am Ende nicht so ist wie gedacht, kann das lokale Arbeitsverzeichnis einfach lokal gelöscht werden. Dann führt man die Schritte einfach nochmal durch.

Zur Überprüfung einmal (in der IDE) schauen, ob der richtige **Branch** namens

hands-on

heruntergeladen wurde. Dies ist sozusagen eine spezielle Variante (oder auch Version), die im Semester genutzt wird.



Ergebnis sollte jetzt sein, dass das Projekt für die LV in der IDE enthalten bzw. geladen ist.

(Optional) Ergänzend für **erfahrenere** Benutzer:

Es kann auch die *Kommandozeile* genutzt werden, um das Repository zu klonen, dazu gibt es ein paar Tipps im folgenden Abschnitt, kann aber für den Moment ignoriert werden, es sei denn, das der Weg über IntelliJ nicht funktionieren sollte.

▼ Aufklappen für Git mit der Kommandozeile ...

- Zuerst prüfen, ob **git** installiert ist. In der Kommandozeile geht dies durch

```
git --version
```

- was zum Beispiel folgende Ausgabe ergibt: `git version 2.39.1`
- Dann in das lokale Arbeitsverzeichnis **wechseln** (oder neu anlegen), d.h. in den Workspace, z.B. nach ...

```
cd D:/Projekte
```

- Innerhalb dieses Verzeichnisses wird das Repository **geklont**, d.h. heruntergeladen. Das Repository lautet:

```
https://github.com/ThorstenEckstein/ma-wwi24ama
```

Dieses Repository wird geklont und damit zu einer lokalen Kopie im **Workspace** mit:

```
git clone https://github.com/ThorstenEckstein/ma-wwi24ama.git -b hands-on ①
```

- ① Der Branch-Name `hands-on` ist hier wichtig, das ist der exakte "Ort" in der Online-Quellcodeverwaltung (Repository), von dem aus heruntergeladen wird!

- Zur Prüfung, auf welchem Branch man sich aktuell befindet, gibt man ein:

```
git branch
```

- Hier sollte jetzt etwas Ähnliches erscheinen. Das Sternchen `*` zeigt an, welcher Branch gerade benutzt wird, das *soll* hier auch so sein:

```
* hands-on
```



... es soll kein Code in das `remote` Repository **gepushed** werden, da es sonst zu Konflikten bei möglicherweise erforderlichen Updates (durch *pullen*) kommen kann!